

# World Model + VLM Scorer для MiniGrid

В работе реализован небольшой model-based RL проект: агент в MiniGrid планирует действия через обученную модель мира и использует VLM-based scorer для оценки будущих состояний относительно текстовой цели `agent at the green goal`.

## 1. Постановка задачи

Требовалось объединить world model и VLM-based scorer для управления агентом в простой среде. Ключевое условие - VLM должен оценивать не только текущее наблюдение, а именно будущие кадры или состояния, полученные через imagined rollouts. Также нужно было сравнить метод с random policy и planner-ом без VLM.

- среда: MiniGrid-Empty-5x5-v0;
- world model: компактная RSSM-style модель в духе PlaNet/Dreamer;
- VLM scorer: CLIP ViT-B/32 через open\_clip;
- planner: MPC/random shooting;
- метрики: success rate, mean return, mean steps.

## 2. Общая архитектура

Пайплайн состоит из пяти шагов: сбор переходов из MiniGrid, обучение модели мира, предрасчёт VLM-score для возможных будущих состояний, MPC-планирование по imagined rollouts и evaluation трёх политик. На каждом реальном шаге среды planner генерирует candidate action sequences, прокручивает их внутри RSSM и выбирает последовательность с максимальным objective.

Objective для варианта World model + VLM включает predicted reward, VLM-score и небольшой goal-distance stabilizer. Последний нужен не как основной метод, а как практическая стабилизация, потому что CLIP обучался на естественных изображениях, а MiniGrid является символическим grid-world.

## 3. Среда и данные

В MiniGrid-Empty-5x5-v0 агент стартует в маленькой комнате и должен дойти до зелёной клетки-цели. Для обучения world model был собран transition dataset: перебирались допустимые позиции агента, четыре направления взгляда и три навигационных действия - left, right, forward. Такой набор покрывает локальную динамику среды и делает обучение компактной модели стабильным.

Итоговый dataset содержит 3240 переходов. Среди них 60 переходов заканчиваются достижением цели и дают положительную награду. Помимо этого были сохранены GIF эпизодов для визуальной проверки поведения политик.

## 4. RSSM world model

RSSM расшифровывается как Recurrent State-Space Model. В такой модели есть детерминированная память  $h_t$  и stochastic latent state  $z_t$ . В полном Dreamer RSSM обычно работает с пиксельными наблюдениями и latent-представлениями. В этом проекте сделана компактная state-based версия: она предсказывает следующий нормализованный state агента, reward и done probability.

Такое упрощение выбрано осознанно: цель задания - показать саму связку world model, imagined rollouts, VLM-score и MPC. Для маленькой среды state-based RSSM позволяет получить стабильный результат без долгого обучения pixel decoder-a.

## 5. VLM scorer

Для VLM scorer используется CLIP ViT-B/32. Все возможные будущие состояния MiniGrid рендерятся как RGB-кадры и сравниваются с текстом `agent at the green goal`. Эти оценки сохраняются в cache, чтобы во время MPC не запускать CLIP заново для каждой candidate trajectory.

Смысловой момент: score применяется к imagined future states, полученным из rollout-a world model. То есть planner выбирает действие не только по текущему кадру, а по оценке возможного будущего.

## 6. Планирование

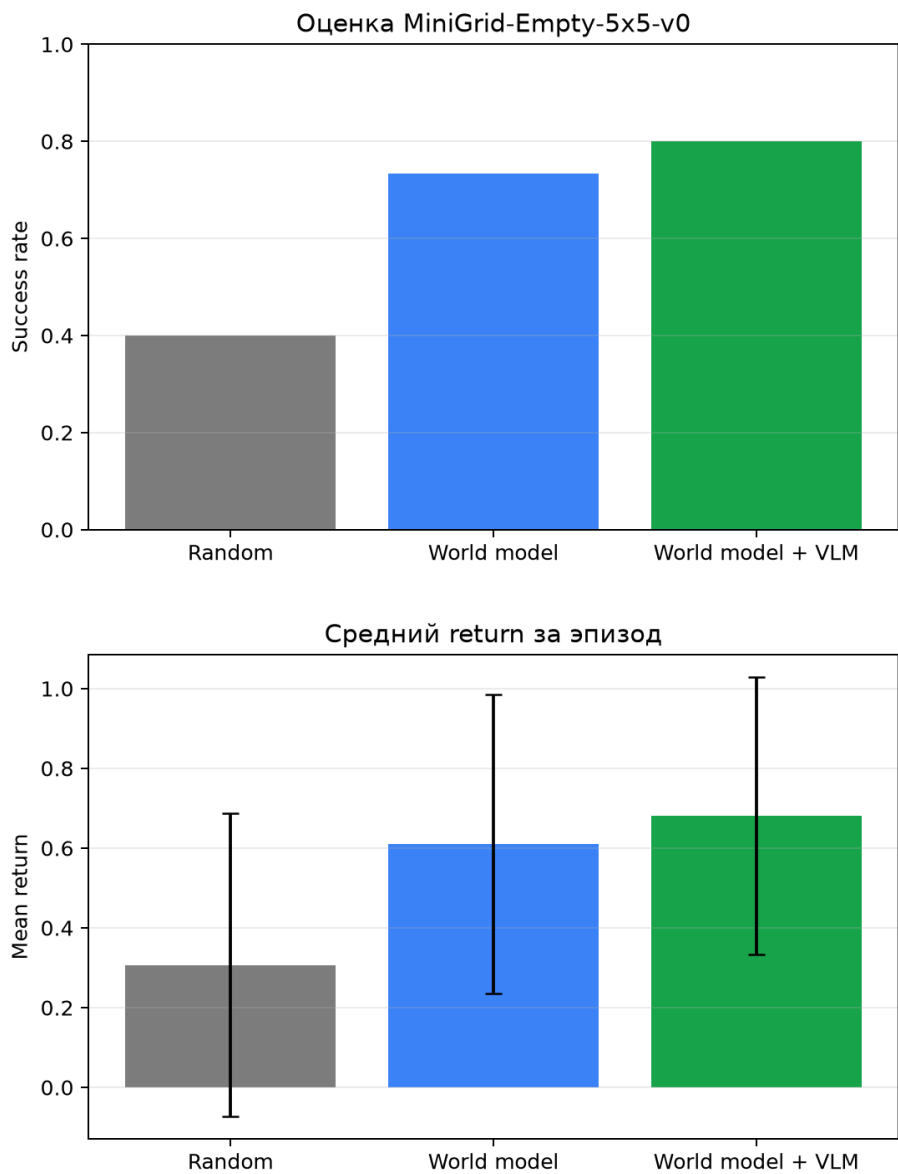
Используется MPC с random shooting. На каждом шаге генерируется 768 случайных последовательностей действий длиной 14. Каждая последовательность прокручивается внутри learned world model. Затем считается objective, выбирается лучшая последовательность, в реальной среде выполняется только её первое действие, после чего процесс повторяется.

## 7. Результаты

| Метод                | Эпизоды | Success rate | Mean return | Mean steps |
|----------------------|---------|--------------|-------------|------------|
| Random               | 30      | 0.40         | 0.307       | 34.3       |
| World model planning | 30      | 0.73         | 0.610       | 24.3       |
| World model + VLM    | 30      | 0.80         | 0.682       | 21.1       |

Оценка проводилась на 30 эпизодах с seeds 0-29. Максимальная длина эпизода - 40 шагов, horizon планирования - 14, число random-shooting candidates - 768.

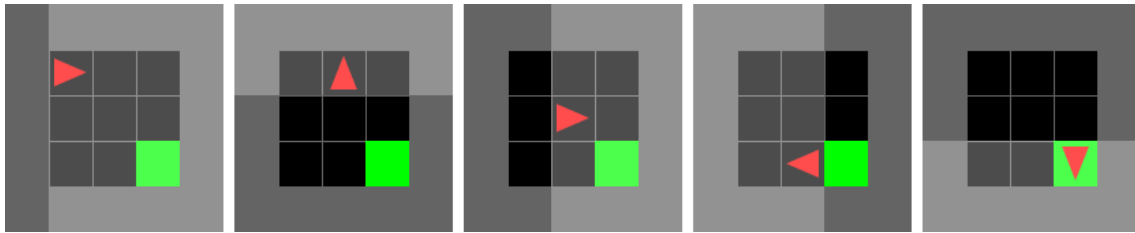
Планирование в learned world model заметно лучше random policy. Добавление VLM-score в этом запуске улучшило success rate с 0.73 до 0.80 и mean return с 0.610 до 0.682. Также World model + VLM в среднем достигает результата быстрее: 21.1 шага против 24.3 у reward-only planner.



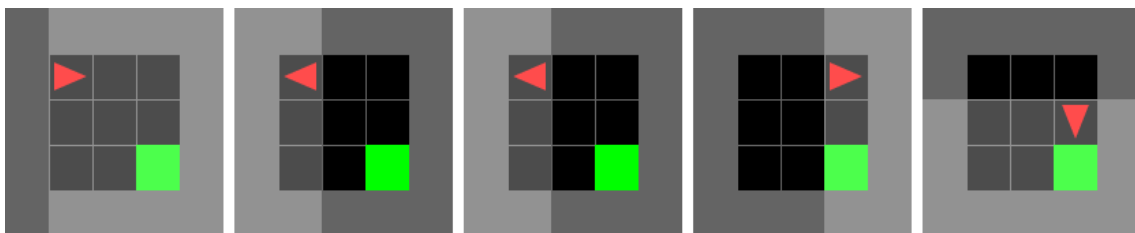
## 8. Визуализация эпизодов

Ниже показаны раскадровки трёх GIF-примеров. Они помогают визуально сравнить поведение random policy, reward-only planner и planner-a с VLM-score. Анимированные версии GIF также сохранены в репозитории.

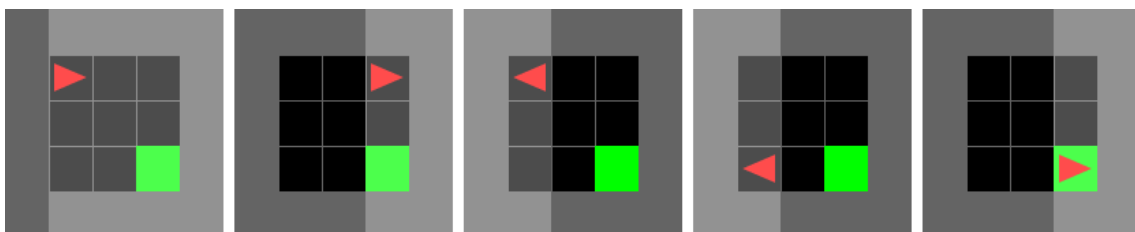
### Random policy



### World model planning



### World model + VLM



## 9. С какими трудностями столкнулись

- CLIP-score оказался шумным на MiniGrid, потому что кадры символические и не похожи на естественные изображения из pretraining.
- Полный pixel-level Dreamer был бы слишком тяжёлым для небольшого учебного демо, поэтому была выбрана компактная state-based RSSM.
- Random shooting требует много candidate trajectories: меньше candidates работает быстрее, но качество planning падает.
- Reward-only planner в простой Empty-среде уже достаточно силён, поэтому VLM-term пришлось взвешивать аккуратно.
- Для PDF GIF нельзя встроить как настоящую анимацию, поэтому в отчёт добавлены раскадровки эпизодов.

## 10. Что можно улучшить дальше

- перейти к DoorKey-8x8, где языковая цель вроде agent next to the key будет полезнее;
- обучить pixel decoder и применять VLM напрямую к decoded imagined frames;
- заменить random shooting на CEM для более эффективного поиска действий;
- добавить uncertainty-aware planning, чтобы planner учитывал неуверенность world model;
- сравнить разные VLM и prompt-формулировки.

## 11. Вывод

Проект показывает рабочую связку world model, imagined rollouts, VLM-based scoring и MPC-planning. Даже в небольшой MiniGrid-среде видно, что learned model позволяет планировать лучше случайной политики, а добавление VLM-score может улучшить выбор действий, если аккуратно учитывать шумность vision-language модели на символических изображениях.